Cactus Creek Clearing — Bid Application Technical Plan v2

Client: Cactus Creek Clearing, Kerrville TX
Purpose: AI powered, map based cedar/brush clearing bid tool with multi-source satellite analysis, soil data integration, 3D visualization, self-improving accuracy, and professional PDF output
Last Updated: April 2026

Table of Contents

1. Executive Summary
2. Application Architecture
3. Map and Drawing Engine
4. Multi-Source Satellite Imagery Pipeline
5. AI Cedar Detection and Density Analysis
6. 3D Terrain and Tree Visualization
7. Soil Data Integration: UC Davis SoilWeb and USDA SDA
8. Bid Rate Engine and Business Logic
9. Self-Improving Feedback Loop
10. PDF Generation
11. Data Model and Database
12. Authentication and Multi User
13. Deployment and Infrastructure
14. UI/UX Best Practices
15. Development Phases
16. Open Questions for Cactus Creek
17. Appendices

---

1. Executive Summary

This is not just a bid calculator. It is an AI powered clearing intelligence platform that gets smarter with every job.

The core idea: draw pasture polygons on a satellite map, and the system automatically pulls imagery from 3 to 5 satellite sources, runs AI analysis to detect cedar density and estimate tree sizes, queries USDA and UC Davis soil databases for terrain difficulty, renders a 3D "god's eye" visualization of the property with modeled trees, calculates a bid using learned historical performance data, and generates a PDF that looks like it came from a company ten times bigger.

After each job, the crew logs actual time per section. That data feeds back into the prediction model. After 6 to 12 months, the system should be able to predict job duration within 10 to 15% accuracy, far better than gut feel estimates.

Why This Is a Competitive Weapon

Most clearing companies bid with a tape measure, a truck drive by, and a gut number. This system gives Cactus Creek:

1. **Satellite verified density analysis** instead of eyeball estimates from the road
2. **Soil aware pricing** that accounts for rock, slope, and drainage automatically

3. **Historical calibration** that eliminates the “we lost money on that job” problem
4. **Professional 3D visualization** in the bid PDF that no competitor can match
5. **Speed**: a bid that currently takes 2 hours drops to 15 minutes

---

2. Application Architecture

Full Stack Overview

Layer	Technology	Purpose
Frontend	Next.js 14+ (App Router)	SSR, React ecosystem, file routing
Map	Mapbox GL JS + @mapbox/mapbox-gl-draw	Satellite view, polygon drawing, 3D terrain
3D Engine	Mapbox GL JS 3D terrain + Three.js (threebox)	God’s eye view with modeled trees
Geo Math	Turf.js	Acreage, centroid, bounding box
AI Vision	Python microservice (FastAPI)	NDVI analysis, cedar detection, tree counting
ML Model	PyTorch or TensorFlow	Tree segmentation, density classification
Satellite Data	Multi-source pipeline (see Section 4)	NAIP, Sentinel-2, Mapbox, Google, Planet
Soil API	UC Davis SoilWeb + USDA SDA REST	Soil series, slope, rock, drainage
Elevation	USGS 3DEP + Mapbox Terrain DEM	Slope analysis, 3D terrain mesh
State	Zustand	Lightweight state management
Styling	Tailwind CSS + shadcn/ui	Professional UI
PDF	Puppeteer (server side)	Full CSS control, map screenshots
Database	Supabase (Postgres + Auth + Storage)	Bids, feedback data, ML training logs
Queue	Supabase Edge Functions or BullMQ	Async satellite processing
Hosting	Vercel (frontend) + Railway (AI service)	Separate compute for ML workloads
Domain	Custom subdomain for CCC	Professional appearance

System Architecture Diagram

```
User draws polygon
       |
       v
  [Next.js Frontend]
       |
       ├── Mapbox GL JS (satellite view + drawing)
       ├── Acreage calc (Turf.js, client side)
       |
       v
  [API Routes / Edge Functions]
       |
       ├── /api/soil ──────> UC Davis SoilWeb API
       |                     USDA SDA REST API
       |
       ├── /api/imagery ───> NAIP ImageServer (0.6m, 4-band)
       |                     Sentinel-2 (10m, multispectral)
       |                     Mapbox satellite tiles
       |                     Google Earth Engine (optional)
```

```
        |                     Planet Labs (optional, paid)
        |
        ├── /api/analyze ──> [Python AI Microservice]
        |                      ├── NDVI computation
        |                      ├── Cedar vs oak vs grass classification
        |                      ├── Tree count and size estimation
        |                      ├── Density heatmap generation
        |                      └── Composite confidence map
        |
        ├── /api/elevation ──> USGS 3DEP
        |                      Mapbox Terrain DEM
        |
        ├── /api/predict ──> [Prediction Engine]
        |                      ├── Historical job data
        |                      ├── Soil difficulty factors
        |                      ├── AI density scores
        |                      └── Time and cost prediction
        |
        ├── /api/pdf ──────> Puppeteer PDF render
        |
        └── /api/feedback ──> Post job review data
                               └── Model retraining pipeline
```

---

3. Map and Drawing Engine

Mapbox Configuration

```
const map = new mapboxgl.Map({
  container: 'map',
  style: 'mapbox://styles/mapbox/satellite-streets-v12',
  center: [-99.1403, 30.0469], // Kerrville TX
  zoom: 14,
  pitch: 0,
  bearing: 0,
});

// Enable 3D terrain (for god's eye view toggle)
map.addSource('mapbox-dem', {
  type: 'raster-dem',
  url: 'mapbox://mapbox.mapbox-terrain-dem-v1',
  tileSize: 512,
  maxzoom: 14,
});

// Toggle terrain on/off
function enable3DTerrain(exaggeration = 1.5) {
  map.setTerrain({ source: 'mapbox-dem', exaggeration });
  map.addLayer({
    id: 'sky',
    type: 'sky',
    paint: {
      'sky-type': 'atmosphere',
      'sky-atmosphere-sun': [0.0, 0.0],
      'sky-atmosphere-sun-intensity': 15,
    },
  });
}
```

Polygon Drawing with Real-Time Acreage

```
import * as turf from '@turf/turf';
```

```
// Calculate acreage from GeoJSON polygon
function calculateAcreage(polygon) {
  const sqMeters = turf.area(polygon);
  return Math.round((sqMeters / 4046.8564224) * 100) / 100;
}

// Calculate bounding box for satellite image requests
function getPolygonBbox(polygon) {
  return turf.bbox(polygon); // [west, south, east, north]
}

// Calculate centroid for soil point queries
function getPolygonCentroid(polygon) {
  const center = turf.centroid(polygon);
  return center.geometry.coordinates; // [lng, lat]
}
```

Best Practices

1. **Default satellite view** with toggleable NAIP and Sentinel overlays
2. **Color code polygons** by vegetation type: green for cedar, brown for oak, orange for mixed, red for full clear
3. **Real time acreage label** updating as user draws each vertex
4. **Multiple polygons per bid** with independent settings per pasture
5. **Polygon editing** (move vertices, add points, delete points) after creation
6. **Offline tile caching** via Mapbox offline APIs for field visits in Hill Country dead zones
7. **Layer toggle panel** to switch between: Satellite, NAIP CIR (false color), NDVI heatmap, Soil map units, Elevation contours, AI density overlay

---

4. Multi-Source Satellite Imagery Pipeline

The Multi-Source Strategy

No single satellite source gives the full picture. By cross referencing 3 to 5 sources, we can build a composite analysis that is significantly more accurate than any one alone. Each source brings different strengths.

Source 1: NAIP (National Agriculture Imagery Program)

Why: Highest resolution free imagery available (0.6m per pixel, some areas 0.3m). 4-band including near infrared (NIR), which is essential for NDVI vegetation analysis. Covers all of Texas.

Resolution: 0.6m (roughly 2 foot pixels). Individual cedar trees are visible at this resolution.

Refresh rate: Every 2 to 3 years. Texas was last flown in 2022 with 2024/2025 coming.

API Access:

```
// USGS NAIP ImageServer (ArcGIS REST)
// Supports ExportImage with bbox, size, and band selection
GET https://imagery.nationalmap.gov/arcgis/rest/services/USGSNAIPImagery/ImageServer/exportImage
  ?bbox=-99.15,30.04,-99.13,30.06
  &bboxSR=4326
  &size=2000,2000
  &imageSR=4326
  &format=png
  &renderingRule={"rasterFunction":"NDVI_Color"}
  &f=image
```

Available rendering templates:

- `NaturalColor` : RGB bands 1,2,3 (standard aerial photo look)
- `FalseColorComposite` : NIR, Red, Green (bands 4,1,2). Cedar shows as bright red/magenta
- `NDVI_Color` : Computed NDVI with color ramp. Dense vegetation = dark green, bare ground = brown

Texas specific NAIP: Also available via the Texas Geographic Information Office (TxGIO) at:

```
https://imagery.geographic.texas.gov/server/rest/services/NAIP/
```

Source 2: Sentinel-2 (European Space Agency, Copernicus Program)

Why: 13 spectral bands including multiple NIR and SWIR bands for advanced vegetation indices. Free and open. 5 day revisit means near current imagery. 20+ year archive for change detection.

Resolution: 10m (visible/NIR), 20m (red edge/SWIR). Too coarse to see individual trees, but excellent for pasture level density classification and seasonal change analysis.

API Access via Sentinel Hub:

```
// Sentinel Hub Process API
const response = await fetch('https://services.sentinel-hub.com/api/v1/process', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': `Bearer ${SENTINEL_HUB_TOKEN}`,
  },
  body: JSON.stringify({
    input: {
      bounds: {
        bbox: [-99.15, 30.04, -99.13, 30.06],
        properties: { crs: 'http://www.opengis.net/def/crs/EPSG/0/4326' },
      },
      data: [{
        type: 'sentinel-2-l2a',
        dataFilter: {
          timeRange: {
            from: '2025-06-01T00:00:00Z',
            to: '2025-09-30T23:59:59Z',
          },
          maxCloudCoverage: 15,
        },
      }],
    },
    output: {
      width: 512,
      height: 512,
      responses: [{ identifier: 'default', format: { type: 'image/tiff' } }],
    },
    evalscript: `
      //VERSION=3
      function setup() {
        return {
          input: ["B04","B08","B11","SCL"],
          output: { bands: 1 }
        };
      }
      function evaluatePixel(sample) {
        // NDVI: (NIR - Red) / (NIR + Red)
        let ndvi = (sample.B08 - sample.B04) / (sample.B08 + sample.B04);
        return [ndvi];
      }
    `,
  }),
});
```

Key bands for cedar detection:

- B04 (Red, 10m): Chlorophyll absorption
- B08 (NIR, 10m): Vegetation reflectance (cedar reflects strongly here)
- B05/B06/B07 (Red Edge, 20m): Species discrimination. Cedars have different red edge profiles than oaks and grasses
- B11 (SWIR, 20m): Moisture content. Cedar (juniper) is more drought tolerant, shows different SWIR than deciduous species

Free access: Sentinel-2 data on AWS (registry.opendata.aws/sentinel-2) is completely free. Sentinel Hub provides a more convenient API with a free tier (30,000 requests/month).

Source 3: Mapbox Satellite

Why: Already integrated for the base map. High resolution (sub-meter in many areas), frequently updated. Good for visual reference and user familiarity.

Access: Mapbox Static Images API or raster tile requests. Already available through the Mapbox GL JS map.

Source 4: Google Earth Engine (Optional but Powerful)

Why: Access to Landsat archive (30+ years), NAIP, Sentinel-2, and MODIS all in one place with server side processing. Can run NDVI, classification, and change detection entirely in the cloud. Free for non-commercial use, paid for commercial.

Use case for this app: Historical change detection. Compare NAIP images from 2014, 2017, 2020, 2022 to see how cedar has encroached over time. This is a powerful sales tool: "Here's how much cedar has grown in the last 8 years if you don't clear it."

```
// Google Earth Engine JavaScript API (runs in GEE Code Editor or via Python API)
// Example: NDVI time series for a polygon
var polygon = ee.Geometry.Polygon([[[-99.15,30.04],[-99.13,30.04],
                                    [-99.13,30.06],[-99.15,30.06]]]);

var ndviCollection = ee.ImageCollection('COPERNICUS/S2_SR_HARMONIZED')
  .filterBounds(polygon)
  .filterDate('2020-01-01', '2025-12-31')
  .filter(ee.Filter.lt('CLOUDY_PIXEL_PERCENTAGE', 15))
  .map(function(image) {
    var ndvi = image.normalizedDifference(['B8', 'B4']).rename('NDVI');
    return image.addBands(ndvi);
  });
```

Source 5: Planet Labs (Premium, Optional)

Why: Daily revisit at 3m resolution. Best temporal coverage of any commercial provider. Useful for monitoring active clearing jobs or seasonal analysis.

Cost: ~$4,000 to $10,000/year for a small area. Only worthwhile if CCC is doing enough volume to justify it. Consider adding later.

Composite Image Processing Pipeline

The goal is to combine these sources into a single "cedar density confidence map" for each polygon.

```
Step 1: Fetch imagery
  ├── NAIP natural color (0.6m) ──> base layer, visual reference
  ├── NAIP CIR / NDVI (0.6m) ───> vegetation presence map
  ├── Sentinel-2 NDVI (10m) ────> broad vegetation density
  ├── Sentinel-2 Red Edge (20m) —> species discrimination
  └── Mapbox satellite ──────────> visual cross reference

Step 2: Align and resample
  ├── Resample all to common grid (1m pixels)
  ├── Georeference align using polygon bbox
  └── Cloud mask (Sentinel SCL band, NAIP visual check)

Step 3: Compute vegetation indices per source
  ├── NDVI from NAIP Band4/Band1
```

```
  ├── NDVI from Sentinel B08/B04
  ├── EVI from Sentinel (enhanced, reduces soil noise)
  ├── NDWI from Sentinel B08/B11 (moisture)
  └── Red Edge Index from Sentinel B07/B05

Step 4: Species classification
  ├── Cedar signature: high NDVI year-round, low SWIR moisture, specific red edge
  ├── Oak signature: seasonal NDVI variation, higher SWIR moisture, different red edge
  ├── Grass: low NDVI in winter, high in spring, very different red edge
  └── ML model refines classification from training data

Step 5: Generate outputs
  ├── Cedar density heatmap (overlay on map)
  ├── Estimated tree count and size distribution
  ├── Confidence score (0 to 100) per grid cell
  └── Composite analysis JSON for bid engine
```

---

5. AI Cedar Detection and Density Analysis

Computer Vision Architecture

The AI pipeline runs as a separate Python microservice (FastAPI on Railway or a dedicated GPU server). The frontend sends polygon coordinates, the service fetches and processes imagery, and returns analysis results.

Phase 1: Rule Based NDVI Analysis (Ship First)

Before building a full ML model, start with rule based vegetation classification using NDVI thresholds. This works surprisingly well for cedar vs bare ground in the Hill Country.

```python
# /ai-service/analysis/ndvi_classifier.py
import numpy as np
from PIL import Image

def classify_vegetation(ndvi_array: np.ndarray) -> dict:
    """
    Classify vegetation from NDVI values.
    NDVI ranges: -1 to +1
    """
    total_pixels = ndvi_array.size

    # Classification thresholds (calibrate with ground truth)
    bare_ground = np.sum(ndvi_array < 0.15) / total_pixels
    sparse_veg = np.sum((ndvi_array >= 0.15) & (ndvi_array < 0.3)) / total_pixels
    moderate_veg = np.sum((ndvi_array >= 0.3) & (ndvi_array < 0.5)) / total_pixels
    dense_veg = np.sum((ndvi_array >= 0.5) & (ndvi_array < 0.7)) / total_pixels
    very_dense = np.sum(ndvi_array >= 0.7) / total_pixels

    # Density score: weighted average, 0-100
    density_score = (
        sparse_veg * 20 +
        moderate_veg * 50 +
        dense_veg * 80 +
        very_dense * 100
    )

    # Map to clearing density categories
    if density_score > 70:
        density_class = 'extreme'
    elif density_score > 50:
        density_class = 'heavy'
    elif density_score > 30:
        density_class = 'moderate'
    else:
        density_class = 'light'
```

```
    return {
        'density_score': round(density_score, 1),
        'density_class': density_class,
        'bare_ground_pct': round(bare_ground * 100, 1),
        'sparse_vegetation_pct': round(sparse_veg * 100, 1),
        'moderate_vegetation_pct': round(moderate_veg * 100, 1),
        'dense_vegetation_pct': round(dense_veg * 100, 1),
        'very_dense_vegetation_pct': round(very_dense * 100, 1),
    }
```

Phase 2: Cedar vs Oak Species Discrimination

Cedar (Ashe juniper, specifically) has a distinct spectral signature compared to live oak and post oak:

1. **Year round greenness:** Cedar is evergreen. In winter NDVI images, anything still bright green in the Hill Country is almost certainly cedar. This is the single most powerful discriminator.
2. **Red edge difference:** Cedar and oak reflect differently in the 700 to 750nm range (Sentinel-2 bands B05/B06/B07). Cedar has a sharper red edge transition.
3. **Texture difference:** In high res NAIP imagery, cedar canopy has a “fuzzy” or “bumpy” texture pattern, while oak canopy is smoother and more rounded.
4. **Shadow patterns:** Cedar typically has a conical shape casting pointed shadows. Oak casts broader, irregular shadows.

Winter vs Summer Comparison:

```
def detect_cedar_by_seasonality(summer_ndvi: np.ndarray, winter_ndvi: np.ndarray) -> np.ndarray:
    """
    Cedar stays green year round. Oaks and grasses lose NDVI in winter.
    Pixels that maintain high NDVI from summer to winter are likely cedar.

    summer_ndvi: NDVI from June to August
    winter_ndvi: NDVI from December to February
    """
    # Seasonal persistence ratio
    # Cedar: winter/summer ratio ~0.85 to 1.0
    # Oak: winter/summer ratio ~0.3 to 0.6
    # Grass: winter/summer ratio ~0.1 to 0.3

    persistence = np.where(summer_ndvi > 0.2, winter_ndvi / summer_ndvi, 0)

    cedar_mask = (persistence > 0.75) & (winter_ndvi > 0.35)
    oak_mask = (persistence > 0.3) & (persistence <= 0.75) & (summer_ndvi > 0.4)
    grass_mask = (persistence <= 0.3) & (summer_ndvi > 0.3)

    return {
        'cedar_mask': cedar_mask,
        'oak_mask': oak_mask,
        'grass_mask': grass_mask,
        'cedar_coverage_pct': round(np.sum(cedar_mask) / cedar_mask.size * 100, 1),
        'oak_coverage_pct': round(np.sum(oak_mask) / oak_mask.size * 100, 1),
    }
```

Phase 3: ML Tree Detection and Size Estimation

Once you have 20 to 30 completed jobs with ground truth data, train a convolutional neural network (CNN) for individual tree detection in NAIP imagery.

Model Architecture:

```
# Tree detection using a U-Net style segmentation model
# Input: 256x256 pixel NAIP patches (RGB + NIR = 4 channels)
# Output: per-pixel classification (background, cedar small, cedar medium, cedar large, oak)
```

```
import torch
import torch.nn as nn

class CedarDetectionModel(nn.Module):
    """
    Semantic segmentation model for cedar tree detection.
    Based on U-Net architecture with pretrained ResNet encoder.
    """
    def __init__(self, num_classes=5, in_channels=4):
        super().__init__()
        # Use segmentation_models_pytorch for production
        # This is a simplified version
        self.encoder = ResNetEncoder(in_channels=in_channels)
        self.decoder = UNetDecoder(num_classes=num_classes)

    def forward(self, x):
        features = self.encoder(x)
        segmentation = self.decoder(features)
        return segmentation

# Classes:
# 0: Background (bare ground, grass, structures)
# 1: Cedar - Small (canopy < 10ft diameter, estimated <8ft tall)
# 2: Cedar - Medium (canopy 10-20ft diameter, estimated 8-15ft tall)
# 3: Cedar - Large (canopy >20ft diameter, estimated >15ft tall)
# 4: Oak / other hardwood
```

Training Data Collection Strategy:

This is the hardest part. You need labeled training data, meaning someone has to look at satellite images and mark which pixels are cedar, oak, grass, etc. Here is how to bootstrap this cheaply:

1. **Manual labeling on first 10 jobs:** Before clearing, walk the pasture with a GPS app (like Avenza Maps) and mark cedar locations. Take photos at each point. Use this to create labeled training patches.
2. **Use the winter/summer NDVI trick** to auto-generate rough labels for cedar (the evergreen classifier from Phase 2). Human operators review and correct these labels.
3. **Before/after comparison:** For each completed job, compare pre-clearing satellite imagery with post-clearing imagery. The difference is exactly where the trees were. This is free training data.
4. **Use an annotation tool** like Label Studio (open source) or Labelbox for the labeling workflow.

Expected timeline to useful ML model:

- 0 to 3 months: Rule based NDVI analysis only (Phase 1 and 2). Still very useful.
- 3 to 6 months: Collect labeled data from completed jobs. Begin training initial model.
- 6 to 12 months: Model becomes accurate enough to replace/augment manual density assessment.
- 12+ months: Model is self-improving with the feedback loop, approaching human-level accuracy.

Tree Count and Size Estimation

At NAIP resolution (0.6m pixels), individual cedar trees are visible as distinct canopy clusters. Use connected component analysis or watershed segmentation to count individual trees and estimate canopy diameter.

```
from scipy import ndimage
import numpy as np

def count_and_size_trees(cedar_mask: np.ndarray, pixel_size_meters: float = 0.6) -> dict:
    """
    Count individual tree canopies and estimate sizes.
    cedar_mask: binary mask where True = cedar pixel
    pixel_size_meters: spatial resolution (0.6m for NAIP)
    """
    # Label connected components (each blob = one tree or cluster)
```

```
    labeled_array, num_features = ndimage.label(cedar_mask)

    trees = []
    for tree_id in range(1, num_features + 1):
        tree_pixels = np.sum(labeled_array == tree_id)
        canopy_area_sqm = tree_pixels * (pixel_size_meters ** 2)
        canopy_diameter_m = 2 * np.sqrt(canopy_area_sqm / np.pi)  # assume circular
        canopy_diameter_ft = canopy_diameter_m * 3.28084

        # Size classification
        if canopy_diameter_ft < 10:
            size_class = 'small'
            estimated_height_ft = canopy_diameter_ft * 0.8
        elif canopy_diameter_ft < 20:
            size_class = 'medium'
            estimated_height_ft = canopy_diameter_ft * 0.7
        else:
            size_class = 'large'
            estimated_height_ft = min(canopy_diameter_ft * 0.6, 30)

        trees.append({
            'id': tree_id,
            'canopy_area_sqft': round(canopy_area_sqm * 10.7639, 1),
            'canopy_diameter_ft': round(canopy_diameter_ft, 1),
            'size_class': size_class,
            'estimated_height_ft': round(estimated_height_ft, 1),
        })

    # Summary statistics
    small_count = sum(1 for t in trees if t['size_class'] == 'small')
    medium_count = sum(1 for t in trees if t['size_class'] == 'medium')
    large_count = sum(1 for t in trees if t['size_class'] == 'large')

    return {
        'total_trees': num_features,
        'small_trees': small_count,
        'medium_trees': medium_count,
        'large_trees': large_count,
        'trees_per_acre': round(num_features / (np.sum(cedar_mask) * pixel_size_meters**2 / 4046.86), 1),
        'total_canopy_coverage_pct': round(np.sum(cedar_mask) / cedar_mask.size * 100, 1),
        'tree_details': trees[:100],  # cap detail list for performance
    }
```

AI Service API Design

```
# /ai-service/main.py
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI(title="Cactus Creek AI Analysis Service")

class AnalysisRequest(BaseModel):
    polygon: dict  # GeoJSON polygon
    vegetation_types: list[str]  # ['cedar', 'oak', 'mixed', 'all']
    include_tree_count: bool = True
    include_3d_data: bool = True
    include_historical: bool = False  # multi-year change detection

class AnalysisResponse(BaseModel):
    density_score: float
    density_class: str
    cedar_coverage_pct: float
    oak_coverage_pct: float
    grass_coverage_pct: float
    bare_ground_pct: float
    tree_count: dict
```

```
    confidence: float  # 0 to 1
    heatmap_url: str  # URL to density heatmap image
    sources_used: list[str]
    processing_time_seconds: float
    # 3D data
    tree_positions: list[dict]  # [{lng, lat, height_ft, canopy_ft, species}]
    terrain_profile: dict

@app.post("/analyze", response_model=AnalysisResponse)
async def analyze_polygon(request: AnalysisRequest):
    # 1. Fetch imagery from all sources
    naip = await fetch_naip_imagery(request.polygon)
    sentinel = await fetch_sentinel_imagery(request.polygon)

    # 2. Compute NDVI from each source
    naip_ndvi = compute_ndvi(naip, source='naip')
    sentinel_ndvi = compute_ndvi(sentinel, source='sentinel')

    # 3. Cedar detection (seasonal comparison)
    winter_sentinel = await fetch_sentinel_imagery(request.polygon, season='winter')
    cedar_map = detect_cedar_by_seasonality(sentinel_ndvi, winter_sentinel)

    # 4. Refine with NAIP high-res
    cedar_refined = refine_with_naip(cedar_map, naip)

    # 5. Tree counting and sizing
    trees = count_and_size_trees(cedar_refined, pixel_size_meters=0.6)

    # 6. Generate heatmap overlay
    heatmap = generate_heatmap(cedar_refined, request.polygon)

    # 7. Prepare 3D tree positions
    tree_positions = extract_tree_positions(trees, request.polygon)

    return AnalysisResponse(...)
```

---

6. 3D Terrain and Tree Visualization

The “Jarvis God’s Eye View”

This is the wow factor feature. The user clicks a “3D View” button, and the map transitions from a flat satellite view to a tilted 3D terrain with modeled cedar trees placed at their detected locations, sized proportionally to estimated canopy diameter and height.

Implementation Strategy

Option A: Mapbox GL JS 3D Terrain + Three.js via Threebox (Recommended)

Mapbox GL JS v2+ has built in 3D terrain support using their DEM tileset. Threebox is a plugin that synchronizes Three.js scene camera with the Mapbox camera, letting you place 3D objects (tree models) on the terrain.

```
// Enable 3D terrain
map.addSource('mapbox-dem', {
  type: 'raster-dem',
  url: 'mapbox://mapbox.mapbox-terrain-dem-v1',
  tileSize: 512,
  maxzoom: 14,
});

map.setTerrain({ source: 'mapbox-dem', exaggeration: 1.3 });

// Add sky for atmosphere
map.addLayer({
  id: 'sky',
  type: 'sky',
```

```
  paint: {
    'sky-type': 'atmosphere',
    'sky-atmosphere-sun': [0.0, 90.0],
    'sky-atmosphere-sun-intensity': 15,
  },
});

// Initialize Threebox for 3D objects
const tb = (window.tb = new Threebox(
  map,
  map.getCanvas().getContext('webgl'),
  {
    defaultLights: true,
    enableSelectingObjects: true,
    enableDraggingObjects: false,
    enableTooltips: true,
  }
));

// Add custom layer for 3D trees
map.addLayer({
  id: '3d-trees',
  type: 'custom',
  renderingMode: '3d',
  onAdd: function () {
    // Tree positions come from AI analysis
    treePositions.forEach((tree) => {
      const treeModel = createTreeModel(tree.species, tree.height_ft, tree.canopy_ft);
      treeModel.setCoords([tree.lng, tree.lat, 0]);
      tb.add(treeModel);
    });
  },
  render: function () {
    tb.update();
  },
});
```

Procedural Tree Models

Instead of loading heavy 3D model files, generate simple procedural tree meshes in Three.js. Cedar and oak have very different shapes.

```
function createTreeModel(species, heightFt, canopyDiameterFt) {
  const heightM = heightFt * 0.3048;
  const canopyRadiusM = (canopyDiameterFt * 0.3048) / 2;

  const group = new THREE.Group();

  if (species === 'cedar') {
    // Cedar/Juniper: conical shape
    const trunkGeom = new THREE.CylinderGeometry(0.15, 0.2, heightM * 0.3, 8);
    const trunkMat = new THREE.MeshPhongMaterial({ color: 0x4a3728 });
    const trunk = new THREE.Mesh(trunkGeom, trunkMat);
    trunk.position.y = heightM * 0.15;

    const canopyGeom = new THREE.ConeGeometry(canopyRadiusM, heightM * 0.8, 8);
    const canopyMat = new THREE.MeshPhongMaterial({
      color: 0x2d5a27,
      flatShading: true,
    });
    const canopy = new THREE.Mesh(canopyGeom, canopyMat);
    canopy.position.y = heightM * 0.5;

    group.add(trunk);
    group.add(canopy);

  } else if (species === 'oak') {
```

```
    // Oak: broad, rounded canopy
    const trunkGeom = new THREE.CylinderGeometry(0.2, 0.3, heightM * 0.4, 8);
    const trunkMat = new THREE.MeshPhongMaterial({ color: 0x5c4033 });
    const trunk = new THREE.Mesh(trunkGeom, trunkMat);
    trunk.position.y = heightM * 0.2;

    const canopyGeom = new THREE.SphereGeometry(canopyRadiusM, 8, 6);
    canopyGeom.scale(1, 0.6, 1); // flatten sphere to dome
    const canopyMat = new THREE.MeshPhongMaterial({
      color: 0x3a7d44,
      flatShading: true,
    });
    const canopy = new THREE.Mesh(canopyGeom, canopyMat);
    canopy.position.y = heightM * 0.55;

    group.add(trunk);
    group.add(canopy);
  }

  return group;
}
```

3D View Features

1. **Orbit camera** with mouse drag to rotate around the property
2. **Zoom to pasture** button that flies the camera to each polygon
3. **Tree color coding:** Green for cedar, darker green for oak, brown for dead/dry
4. **Size proportional:** Tree heights and canopy diameters match AI estimates
5. **Density heatmap on terrain:** Semi transparent color overlay on the 3D ground showing vegetation density
6. **Toggle tree visibility** by species (show only cedar, only oak, or all)
7. **Polygon outlines** extruded slightly above terrain so they don't clip into hills
8. **Sun position** matching actual sun angle for the property location (shadow realism)
9. **Screenshot capture** of the 3D view for inclusion in the PDF bid

Elevation Data Sources

Mapbox Terrain DEM:

- Built into Mapbox GL JS (mapbox.mapbox-terrain-dem-v1)
- Resolution: ~30m globally, finer in some areas
- Sufficient for 3D terrain rendering

USGS 3DEP (3D Elevation Program):

- Higher resolution (1m to 10m in many areas)
- Available as point cloud (LiDAR) and DEM rasters
- API: `https://elevation.nationalmap.gov/arcgis/rest/services/3DEPElevation/ImageServer`
- Use for precise slope calculations, better than Mapbox DEM

```
// USGS 3DEP Elevation query
GET https://epqs.nationalmap.gov/v1/json
  ?x=-99.14
  &y=30.05
  &wkid=4326
  &units=Feet
  &includeDate=false
```

Performance Considerations

Rendering hundreds or thousands of 3D tree models can crush performance. Strategies:

1. **Level of Detail (LOD):** When zoomed out, replace individual tree models with a simplified representation (colored dots or instanced low-poly shapes). Only render full tree models when zoomed close.
2. **Instanced rendering:** Use `THREE.InstancedMesh` instead of individual meshes. This renders thousands of identical geometries in a single draw call.
3. **View frustum culling:** Only render trees visible on screen.
4. **Progressive loading:** Render the terrain first, then add trees incrementally.
5. **GPU limits:** Target 60 FPS. If performance drops below 30 FPS, reduce tree count by showing only every Nth tree.

```
// Instanced rendering for thousands of trees
function createInstancedTrees(trees, geometry, material) {
  const mesh = new THREE.InstancedMesh(geometry, material, trees.length);
  const matrix = new THREE.Matrix4();

  trees.forEach((tree, i) => {
    const position = tb.utils.lnglat([tree.lng, tree.lat]);
    const scale = tree.height_ft / 20; // normalize
    matrix.makeScale(scale, scale, scale);
    matrix.setPosition(position.x, position.y, position.z);
    mesh.setMatrixAt(i, matrix);
  });

  mesh.instanceMatrix.needsUpdate = true;
  return mesh;
}
```

---

7. Soil Data Integration: UC Davis SoilWeb and USDA SDA

Dual Source Strategy

Use BOTH UC Davis SoilWeb and USDA SDA together. They access the same underlying SSURGO database but offer different strengths.

UC Davis SoilWeb

The UC Davis California Soil Resource Lab wraps SSURGO data in a more accessible interface with pre-computed aggregations and properties.

SoilWeb Point Query:

```
GET https://casoilresource.lawr.ucdavis.edu/soil_web/query.php
  ?lon=-99.14
  &lat=30.05
```

This returns the map unit and component data for that point, including soil series name, taxonomic classification, and a link to the full Soil Data Explorer entry for the series.

SoilWeb Properties Grid: UC Davis publishes pre-aggregated 800m grids of common soil properties. These can be fetched as WMS tiles for overlay:

- Available Water Storage (0 to 100cm)
- Organic Matter % (surface)
- Clay % (surface)
- pH (surface)
- Depth to Restrictive Layer

- Drainage Class
- Hydrologic Group
- Land Capability Class
- Soil Taxonomy (Order, Suborder, Great Group)
- Flooding and Ponding Frequency

Soil Data Explorer (SDE) by Series:

```
GET https://casoilresource.lawr.ucdavis.edu/sde/?series=tarrant
```

Returns the complete profile for a soil series, including horizon by horizon data (depths, textures, rock fragments, pH, etc.), typical landform, parent material, drainage, and associated soils. This is useful for building the soil data card in the bid.

USDA Soil Data Access (SDA) REST API

More powerful for polygon-based queries. Send SQL against the full SSURGO database.

Base Endpoint:

```
POST https://SDMDataAccess.sc.egov.usda.gov/Tabular/post.rest
Content-Type: application/json

{
  "query": "YOUR SQL HERE",
  "format": "JSON"
}
```

Combined Query Strategy

For each drawn polygon:

1. **UC Davis SoilWeb point query** at the polygon centroid for quick soil series identification
2. **USDA SDA polygon query** using the full polygon geometry for complete map unit intersection and component weighted data
3. **UC Davis SDE lookup** for the dominant soil series to get the full profile description
4. **SDA rock fragments query** for surface horizon rock volume percentage
5. **SDA restrictive layer query** for depth to bedrock

Key Soil Properties and Their Clearing Impact

Property	Source	Column/Endpoint	Clearing Impact
Soil series name	UC Davis SoilWeb	Point query	Operator recognition, documentation
Slope %	SDA	`slope_r` in component	Equipment speed, erosion risk
Rock fragment %	SDA	`fragvol_r` in chfrags	Equipment wear, mulcher damage
Depth to bedrock (cm)	SDA	`resdept_r` in corestrictions	Root removal depth, equipment limitation
Drainage class	SDA	`drainagecl` in component	Seasonal access, bogging risk
Hydrologic group	SDA	`hydgrp` in component	Runoff behavior post-clearing
Flooding frequency	SDA	`flodfreqcl` in component	Scheduling constraint
Clay %	UC Davis grid	Properties WMS	Soil behavior when wet (sticky, impassable)
Land capability class	SDA	`nirrcapcl` in component	General terrain difficulty indicator

Taxonomic subgroup	SDA	`taxsubgrp` in component	Deep soil classification

SDA Query: Full Polygon Intersection

```
DECLARE @aoi GEOMETRY;
SET @aoi = geometry::STGeomFromText(
  'POLYGON((-99.15 30.05, -99.14 30.05, -99.14 30.04, -99.15 30.04, -99.15 30.05))',
  4326
);

-- Main component data
SELECT
  mu.mukey, mu.muname, mu.mukind,
  co.cokey, co.compname, co.comppct_r,
  co.slope_r, co.slope_l, co.slope_h,
  co.drainagecl, co.hydgrp,
  co.nirrcapcl, co.flodfreqcl,
  co.taxorder, co.taxsuborder, co.taxsubgrp
FROM
  SDA_Get_Mukey_from_intersection_with_WktWgs84(@aoi) AS mk
  INNER JOIN mapunit AS mu ON mk.mukey = mu.mukey
  INNER JOIN component AS co ON mu.mukey = co.mukey
WHERE
  co.majcompflag = 'Yes'
ORDER BY
  co.comppct_r DESC;
```

Soil to Difficulty Multiplier

```
function calculateSoilDifficulty(soilData: SoilRecord): number {
  let multiplier = 1.0;

  // Slope
  if (soilData.slope_r > 20) multiplier *= 1.5;
  else if (soilData.slope_r > 12) multiplier *= 1.25;
  else if (soilData.slope_r > 5) multiplier *= 1.1;

  // Rock fragments
  if (soilData.fragvol_r > 50) multiplier *= 1.4;
  else if (soilData.fragvol_r > 25) multiplier *= 1.2;
  else if (soilData.fragvol_r > 10) multiplier *= 1.1;

  // Drainage
  if (soilData.drainagecl === 'Poorly drained') multiplier *= 1.3;
  else if (soilData.drainagecl === 'Somewhat poorly drained') multiplier *= 1.15;

  // Bedrock depth
  if (soilData.resdept_r && soilData.resdept_r < 25) multiplier *= 1.3;
  else if (soilData.resdept_r && soilData.resdept_r < 50) multiplier *= 1.15;

  return Math.round(multiplier * 100) / 100;
}
```

SDA WMS Soil Map Overlay

Display soil map unit boundaries on the map as a toggleable layer:

```
map.addSource('soil-units', {
  type: 'raster',
  tiles: [
    'https://SDMDataAccess.sc.egov.usda.gov/Spatial/SDM.wms' +
    '?SERVICE=WMS&VERSION=1.1.1&REQUEST=GetMap' +
    '&LAYERS=mapunitpoly' +
    '&BBOX={bbox-epsg-3857}' +
```

```
    '&SRS=EPSG:3857' +
    '&WIDTH=256&HEIGHT=256' +
    '&FORMAT=image/png' +
    '&TRANSPARENT=true'
  ],
  tileSize: 256,
});

map.addLayer({
  id: 'soil-overlay',
  type: 'raster',
  source: 'soil-units',
  paint: { 'raster-opacity': 0.4 },
  layout: { visibility: 'none' }, // toggle on/off
});
```

Best Practices for Soil Data

1. **Cache everything.** SSURGO refreshes once per year. Cache by polygon hash.
2. **Weight by component percentage.** A drawn pasture often overlaps multiple map units with multiple components. Use `comppct_r` to weight the difficulty multiplier.
3. **Show the data, don't hide it.** Operators know their land. Display soil series name, slope range, rock %, and depth to bedrock in a clear panel. Let them override the auto multiplier.
4. **Fail gracefully.** Both SDA and SoilWeb are government services. If they are down, skip the auto soil analysis and let the user set difficulty manually.
5. **Hill Country defaults.** Kerrville area is predominantly shallow, rocky, calcareous soils (Tarrant, Brackett, Eckrant, Real, Comfort). Default to "moderate to difficult" unless data says otherwise.

---

8. Bid Rate Engine and Business Logic

Rate Card Structure

```
interface RateCard {
  baseRates: {
    cedarOnly: number;       // $/acre
    oakOnly: number;
    mixedBrush: number;
    fullClear: number;
    selectiveThin: number;
    mesquiteOnly: number;
  };
  densityMultipliers: {
    light: number;     // e.g., 0.75
    moderate: number;  // e.g., 1.0
    heavy: number;     // e.g., 1.35
    extreme: number;   // e.g., 1.65
  };
  terrainMultipliers: {
    flat: number;
    moderateSlope: number;
    steep: number;
    steepRocky: number;
  };
  disposalAdders: {
    stackAndBurn: number;  // $/acre
    mulchInPlace: number;
    haulOff: number;
    chipAndSpread: number;
  };
  fixedItems: {
```

```
    mobilization: number;
    burnPermit: number;
    fenceProtectionPerFoot: number;
    waterBarPerUnit: number;
  };
  timeEstimates: Record<string, number>;  // hrs/acre by type+density combo
  minimumBidAmount: number;
}
```

Bid Calculation

```
pastureCost =
  acreage
  × baseRate[vegType]
  × densityMultiplier[density]
  × max(terrainMultiplier, soilMultiplier)
  + (acreage × disposalAdder[method])

// If AI analysis available, blend AI density with manual selection
effectiveDensity = (aiDensityScore * aiWeight) + (manualDensity * (1 - aiWeight))
// aiWeight starts at 0.3, increases to 0.7 as model accuracy improves

totalBid = sum(pastureCosts) + mobilization + permits + customItems
```

Prediction Engine (After Feedback Data Exists)

Once you have 20+ completed jobs with actual time data:

```
def predict_job_duration(
    acreage: float,
    density_score: float,
    soil_multiplier: float,
    vegetation_type: str,
    equipment_type: str,
    historical_jobs: list[dict],
) -> dict:
    """
    Predict hours per acre based on similar historical jobs.
    Uses weighted k-nearest-neighbors on feature space.
    """
    # Feature vector for this job
    features = np.array([acreage, density_score, soil_multiplier])

    # Find similar historical jobs
    similarities = []
    for job in historical_jobs:
        if job['vegetation_type'] == vegetation_type:
            job_features = np.array([
                job['acreage'],
                job['density_score'],
                job['soil_multiplier'],
            ])
            distance = np.linalg.norm(features - job_features)
            similarities.append((distance, job['actual_hrs_per_acre']))

    # Weighted average of top 5 similar jobs
    similarities.sort(key=lambda x: x[0])
    top_k = similarities[:5]

    if len(top_k) == 0:
        return {'predicted_hrs_per_acre': None, 'confidence': 0}

    weights = [1 / (d + 0.01) for d, _ in top_k]
    weight_sum = sum(weights)
    predicted = sum(w * hrs for (_, hrs), w in zip(top_k, weights)) / weight_sum
```

```
    return {
        'predicted_hrs_per_acre': round(predicted, 2),
        'predicted_total_hours': round(predicted * acreage, 1),
        'predicted_days': round(predicted * acreage / 9, 1),  # 9 hr work day
        'confidence': min(len(top_k) / 5, 1.0),
        'similar_jobs_used': len(top_k),
    }
```

---

9. Self-Improving Feedback Loop

This is the most important long term feature. The idea is simple: after every completed job, the crew records how long each section actually took. That real world data flows back into the system to calibrate future predictions.

Post-Job Review Interface

After a job is marked complete, the app presents a review form for each pasture:

```
interface PostJobReview {
  pastureId: string;

  // Time tracking
  actualHoursTotal: number;
  actualHoursPerAcre: number;  // computed

  // Equipment used
  equipmentUsed: string[];  // ['forestry_mulcher', 'chainsaw_crew', 'dozer']
  crewSize: number;

  // Conditions encountered
  actualDensity: 'lighter_than_expected' | 'as_expected' | 'heavier_than_expected';
  actualTerrain: 'easier_than_expected' | 'as_expected' | 'harder_than_expected';
  surprises: string;  // free text: "Hit limestone shelf at 6 inches", "Creek crossing was impassable", etc.

  // Accuracy scoring
  bidAccuracyRating: 1 | 2 | 3 | 4 | 5;  // how close was the bid to reality
  densityEstimateAccuracy: 1 | 2 | 3 | 4 | 5;
  soilEstimateAccuracy: 1 | 2 | 3 | 4 | 5;

  // Photos
  beforePhotos: string[];  // URLs
  afterPhotos: string[];

  // Weather impact
  weatherDelayHours: number;
  weatherNotes: string;
}
```

Feedback Data Processing

```
def process_job_feedback(review: PostJobReview, original_bid: Bid, pasture: Pasture):
    """
    Compare predicted vs actual performance and update calibration data.
    """
    predicted_hrs_per_acre = pasture.estimated_hrs_per_acre
    actual_hrs_per_acre = review.actualHoursTotal / pasture.acreage

    # Accuracy metrics
    error_pct = abs(predicted_hrs_per_acre - actual_hrs_per_acre) / actual_hrs_per_acre * 100
    direction = 'over' if predicted_hrs_per_acre > actual_hrs_per_acre else 'under'

    # Store calibration record
    calibration = {
```

```
        'pasture_id': pasture.id,
        'vegetation_type': pasture.vegetation_type,
        'density_class': pasture.density,
        'ai_density_score': pasture.ai_density_score,
        'terrain_class': pasture.terrain,
        'soil_multiplier': pasture.soil_multiplier,
        'acreage': pasture.acreage,
        'predicted_hrs_per_acre': predicted_hrs_per_acre,
        'actual_hrs_per_acre': actual_hrs_per_acre,
        'error_pct': error_pct,
        'error_direction': direction,
        'equipment_used': review.equipmentUsed,
        'crew_size': review.crewSize,
        'weather_delay_hours': review.weatherDelayHours,
        'soil_series': pasture.soil_data.get('compname'),
        'slope_r': pasture.soil_data.get('slope_r'),
        'rock_fragment_pct': pasture.soil_data.get('fragvol_r'),
        'notes': review.surprises,
    }

    save_calibration_record(calibration)

    # Check if we have enough data to retrain
    total_records = count_calibration_records()
    if total_records >= 20 and total_records % 5 == 0:
        trigger_model_retrain()

    return {
        'error_pct': round(error_pct, 1),
        'direction': direction,
        'total_calibration_records': total_records,
    }
```

Model Retraining Pipeline

```
def retrain_prediction_model():
    """
    Retrain the time prediction model using all calibration data.
    Runs automatically after every 5 new completed jobs.
    """
    records = load_all_calibration_records()

    # Feature engineering
    features = []
    targets = []
    for r in records:
        features.append([
            r['acreage'],
            r['ai_density_score'],
            r['soil_multiplier'],
            encode_vegetation_type(r['vegetation_type']),
            encode_terrain(r['terrain_class']),
            r['slope_r'] or 3.0,
            r['rock_fragment_pct'] or 10.0,
            r['crew_size'],
        ])
        # Target: actual hours per acre (excluding weather delays)
        net_hours = r['actual_hrs_per_acre']
        if r['weather_delay_hours'] > 0:
            net_hours -= r['weather_delay_hours'] / r['acreage']
        targets.append(max(net_hours, 0.5))

    X = np.array(features)
    y = np.array(targets)

    # Train model (start with gradient boosting, upgrade to neural net later)
```

```
    from sklearn.ensemble import GradientBoostingRegressor
    from sklearn.model_selection import cross_val_score

    model = GradientBoostingRegressor(
        n_estimators=100,
        max_depth=4,
        learning_rate=0.1,
        random_state=42,
    )

    # Cross validation score
    scores = cross_val_score(model, X, y, cv=min(5, len(records)), scoring='neg_mean_absolute_error')
    mae = -scores.mean()

    # Train on full data
    model.fit(X, y)

    # Save model
    save_model(model, version=len(records))

    # Log performance
    log_model_performance({
        'training_records': len(records),
        'mae_hrs_per_acre': round(mae, 3),
        'feature_importances': dict(zip(FEATURE_NAMES, model.feature_importances_)),
    })

    return {
        'status': 'retrained',
        'records_used': len(records),
        'mae': round(mae, 3),
    }
```

Accuracy Dashboard

Show the operator how the system is improving over time:

```
interface AccuracyDashboard {
  totalJobsCompleted: number;
  averageErrorPct: number;  // current average prediction error
  errorTrend: number[];  // rolling average error over time
  bestPredictedCategory: string;  // "cedar/moderate density" has lowest error
  worstPredictedCategory: string;  // "mixed/steep terrain" has highest error
  modelVersion: number;
  lastRetrainDate: string;

  // Before/after the AI
  manualEstimateAvgError: number;  // how accurate were the old gut-feel estimates
  aiEstimateAvgError: number;  // how accurate is the AI
  improvementPct: number;
}
```

Feedback Loop Timeline

Month	Data	Accuracy	Notes
0 to 3	0 to 10 jobs	Manual + NDVI rules	Bid accuracy ~40 to 60% (gut feel baseline)
3 to 6	10 to 25 jobs	First ML model	Bid accuracy ~25 to 35% error
6 to 9	25 to 50 jobs	Model v2 + tree detection	Bid accuracy ~15 to 25% error
9 to 12	50 to 75 jobs	Mature model	Bid accuracy ~10 to 15% error
12+	75+ jobs	Self-calibrating	Bid accuracy <10% error, seasonal adjustments

Key insight: The system doesn't need to be perfect on day one. It just needs to be better than a gut estimate, and then get better every month. After a year of collecting feedback data, it will be dramatically more accurate than any competitor's bidding process.

Before/After Photo Comparison

Store drone or phone photos taken before and after clearing. These serve triple duty:

1. **Training data for the AI** (before photos = labeled vegetation, after photos = ground truth for what was there)
2. **Client marketing** (before/after galleries on CCC's website)
3. **Dispute resolution** (if a client claims work wasn't completed, photos prove it)

```
interface PhotoPair {
  pastureId: string;
  location: { lat: number; lng: number };
  beforeUrl: string;
  afterUrl: string;
  capturedAt: string;
  heading: number;  // compass direction camera was pointing
}
```

---

10. PDF Generation

Enhanced PDF with AI Analysis

The bid PDF now includes:

Page 1: Cover

- Company logo, "Clearing Proposal" title, client info, date, bid number

Page 2: Property Overview

- Satellite map with all pasture polygons
- Total acreage, pasture count, estimated duration
- AI confidence score and data sources used

Page 3: AI Analysis Summary (NEW)

- Cedar density heatmap overlaid on satellite image
- Tree count summary: X total trees, Y small/Z medium/W large
- Species breakdown pie chart: cedar %, oak %, grass %, bare %
- Soil summary table: dominant series, slope range, rock %, drainage
- 3D terrain screenshot showing modeled trees (the wow factor)

Page 4+: Pasture Detail (per pasture)

- Zoomed satellite image with AI density overlay
- Pasture stats table (acreage, veg type, density, terrain, soil)
- Line item cost breakdown with multipliers shown
- Subtotal

Final Page: Bid Summary

- Pasture subtotals
- Additional items (mobilization, permits)

- Total bid (large, prominent)
- Estimated timeline (range)
- If feedback data exists: “Based on X similar completed projects”
- Terms, signature lines, validity date

Implementation

Same Puppeteer approach as original plan. The PDF render route now includes AI imagery:

```
// /app/api/pdf/route.ts
export async function POST(req: NextRequest) {
  const { bidId } = await req.json();

  // Generate 3D screenshot before PDF render
  const screenshot3d = await capture3DView(bidId);
  const heatmapImages = await getHeatmapImages(bidId);

  // Store images in temp storage for the PDF route to access
  await storeTemporaryImages(bidId, { screenshot3d, heatmapImages });

  // Render PDF via Puppeteer
  const browser = await puppeteer.launch({ headless: true, args: ['--no-sandbox'] });
  const page = await browser.newPage();
  await page.goto(`${APP_URL}/pdf-render/${bidId}`, { waitUntil: 'networkidle0' });

  const pdf = await page.pdf({
    format: 'Letter',
    printBackground: true,
    margin: { top: '0.5in', bottom: '0.75in', left: '0.5in', right: '0.5in' },
  });

  await browser.close();
  return new NextResponse(pdf, {
    headers: {
      'Content-Type': 'application/pdf',
      'Content-Disposition': `attachment; filename="CCC-Bid-${bidId}.pdf"`,
    },
  });
}
```

---

11. Data Model and Database

Expanded Schema

```
-- Companies (same as v1)
CREATE TABLE companies (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  logo_url TEXT,
  address TEXT, phone TEXT, email TEXT, website TEXT,
  license_number TEXT, insurance_info TEXT,
  terms_and_conditions TEXT,
  rate_card JSONB NOT NULL DEFAULT '{}',
  created_at TIMESTAMPTZ DEFAULT now()
);

-- Users
CREATE TABLE users (
  id UUID PRIMARY KEY REFERENCES auth.users(id),
  company_id UUID REFERENCES companies(id),
  full_name TEXT NOT NULL,
```

```
  role TEXT DEFAULT 'operator',
  phone TEXT,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- Clients
CREATE TABLE clients (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID REFERENCES companies(id),
  name TEXT NOT NULL,
  email TEXT, phone TEXT, address TEXT, notes TEXT,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- Bids
CREATE TABLE bids (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID REFERENCES companies(id),
  client_id UUID REFERENCES clients(id),
  created_by UUID REFERENCES users(id),
  bid_number TEXT NOT NULL,
  status TEXT DEFAULT 'draft',
  property_name TEXT,
  property_address TEXT,
  property_center JSONB,
  map_zoom REAL,
  total_acreage REAL,
  total_amount NUMERIC(12,2),
  estimated_days_low REAL,
  estimated_days_high REAL,
  mobilization_fee NUMERIC(10,2) DEFAULT 0,
  burn_permit_fee NUMERIC(10,2) DEFAULT 0,
  custom_line_items JSONB DEFAULT '[]',
  contingency_pct REAL DEFAULT 0,
  discount_pct REAL DEFAULT 0,
  notes TEXT,
  valid_until DATE,
  rate_card_snapshot JSONB,
  -- NEW: AI analysis metadata
  ai_analysis_id UUID,
  ai_confidence_score REAL,
  prediction_model_version INT,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- Pastures
CREATE TABLE pastures (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  bid_id UUID REFERENCES bids(id) ON DELETE CASCADE,
  name TEXT NOT NULL,
  sort_order INT DEFAULT 0,
  polygon JSONB NOT NULL,
  acreage REAL NOT NULL,
  centroid JSONB,
  vegetation_type TEXT NOT NULL,
  density TEXT NOT NULL,
  terrain TEXT NOT NULL,
  disposal_method TEXT NOT NULL,
  -- Soil data
  soil_data JSONB,
  soil_multiplier REAL DEFAULT 1.0,
  soil_multiplier_override REAL,
  -- NEW: AI analysis data
  ai_density_score REAL,
  ai_cedar_coverage_pct REAL,
  ai_oak_coverage_pct REAL,
```

```
  ai_tree_count JSONB,  -- {total, small, medium, large}
  ai_heatmap_url TEXT,
  ai_tree_positions JSONB,  -- array of {lng, lat, height, canopy, species}
  -- Predictions
  estimated_hrs_per_acre REAL,
  predicted_hrs_per_acre REAL,  -- from ML model
  prediction_confidence REAL,
  -- Financials
  subtotal NUMERIC(10,2),
  notes TEXT,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- NEW: Calibration records (feedback loop)
CREATE TABLE calibration_records (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID REFERENCES companies(id),
  bid_id UUID REFERENCES bids(id),
  pasture_id UUID REFERENCES pastures(id),
  -- Job context
  vegetation_type TEXT,
  density_class TEXT,
  terrain_class TEXT,
  acreage REAL,
  soil_series TEXT,
  slope_r REAL,
  rock_fragment_pct REAL,
  soil_multiplier REAL,
  ai_density_score REAL,
  -- Predictions
  predicted_hrs_per_acre REAL,
  predicted_cost_per_acre REAL,
  -- Actuals
  actual_hours_total REAL,
  actual_hrs_per_acre REAL,
  actual_cost_total REAL,
  -- Crew and equipment
  equipment_used TEXT[],
  crew_size INT,
  weather_delay_hours REAL,
  -- Accuracy assessment
  error_pct REAL,
  error_direction TEXT,  -- 'over' or 'under'
  density_accuracy INT,  -- 1-5
  soil_accuracy INT,  -- 1-5
  overall_accuracy INT,  -- 1-5
  notes TEXT,
  -- Photos
  before_photos TEXT[],
  after_photos TEXT[],
  created_at TIMESTAMPTZ DEFAULT now()
);

-- NEW: Model performance log
CREATE TABLE model_performance_log (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID REFERENCES companies(id),
  model_version INT,
  training_records INT,
  mae_hrs_per_acre REAL,
  feature_importances JSONB,
  retrained_at TIMESTAMPTZ DEFAULT now()
);

-- NEW: Satellite imagery cache
CREATE TABLE imagery_cache (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
  polygon_hash TEXT NOT NULL,
  source TEXT NOT NULL,  -- 'naip', 'sentinel', 'mapbox'
  imagery_date DATE,
  ndvi_data JSONB,
  analysis_result JSONB,
  image_url TEXT,
  fetched_at TIMESTAMPTZ DEFAULT now(),
  UNIQUE(polygon_hash, source)
);

-- Soil cache
CREATE TABLE soil_cache (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  polygon_hash TEXT UNIQUE NOT NULL,
  soil_data JSONB NOT NULL,
  queried_at TIMESTAMPTZ DEFAULT now()
);

-- PDF versions
CREATE TABLE pdf_versions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  bid_id UUID REFERENCES bids(id) ON DELETE CASCADE,
  version INT NOT NULL,
  file_url TEXT NOT NULL,
  generated_at TIMESTAMPTZ DEFAULT now()
);

-- Indexes
CREATE INDEX idx_calibration_company ON calibration_records(company_id);
CREATE INDEX idx_calibration_veg_type ON calibration_records(vegetation_type);
CREATE INDEX idx_calibration_density ON calibration_records(density_class);
CREATE INDEX idx_imagery_hash ON imagery_cache(polygon_hash);
CREATE INDEX idx_soil_hash ON soil_cache(polygon_hash);
CREATE INDEX idx_bids_company ON bids(company_id);
CREATE INDEX idx_bids_status ON bids(status);
CREATE INDEX idx_pastures_bid ON pastures(bid_id);
```

---

12. Authentication and Multi User

Same as v1: Supabase Auth with email/password or magic link. Owner/Operator/Viewer roles. Row Level Security on all tables.

---

13. Deployment and Infrastructure

Split Architecture

Service	Host	Why
Next.js frontend + API routes	Vercel	Zero config, fast deploys, edge functions
AI/ML microservice (Python)	Railway or Render	GPU access for inference, larger memory
Database	Supabase	Managed Postgres, Auth, Storage
Image processing queue	Railway (BullMQ) or Supabase Edge Functions	Async satellite fetching and analysis
Model storage	Supabase Storage or S3	Store trained model artifacts

Cost Estimate

Service	Monthly Cost

Vercel Pro	$20
Supabase Pro	$25
Railway (AI service)	$20 to $50
Mapbox (under 50k loads)	Free
Sentinel Hub (free tier)	$0
NAIP / SDA / UC Davis	Free (government)
Custom domain	~$12/year
Total	**~$75 to $100/month**

Planet Labs or commercial satellite imagery would add $4,000+ per year. Only add if volume justifies it.

---

14. UI/UX Best Practices

Same design principles as v1 (large touch targets, high contrast, minimal typing, auto save), plus:

1. **AI loading state:** Satellite analysis takes 10 to 30 seconds. Show a progress bar with steps: "Fetching NAIP imagery... Computing NDVI... Detecting cedar... Counting trees..."
2. **3D toggle button:** One click switches between 2D satellite and 3D terrain view with animated camera transition
3. **Density heatmap slider:** Opacity control for the AI density overlay
4. **Confidence indicator:** Show green/yellow/red badge next to AI analysis results based on confidence score
5. **Before/after slider:** On the accuracy dashboard, drag a slider across before and after photos of completed jobs
6. **Accuracy trend chart:** Line graph showing prediction error % decreasing over time

Color Palette (Earth Tones for Hill Country)

Use	Color	Hex
Primary	Deep sage green	#4A6741
Secondary	Warm tan / caliche	#C4A76C
Danger / alert	Burnt orange	#CC5500
Background	Off white	#F5F2EB
Text	Charcoal	#2D2D2D
Cedar overlay	Green	#2D5A27 @ 40%
Oak overlay	Brown green	#6B8E23 @ 40%
Density high	Red	#FF0000 @ 30%
Density low	Blue	#0000FF @ 30%

---

15. Development Phases

Phase 1: Core MVP (2 to 3 weeks)

- Next.js scaffold with Tailwind, shadcn/ui

- Mapbox with satellite view and polygon drawing
- Acreage calc (Turf.js)
- Pasture form: veg type, density, terrain, disposal
- Basic rate engine with hardcoded defaults
- Bid summary (on screen total)
- Supabase: bids, pastures, basic auth

Ship: Working bid calculator with map.

Phase 2: Soil Integration (1 to 2 weeks)

- UC Davis SoilWeb point query integration
- USDA SDA polygon query integration
- Soil data display per pasture
- Auto soil difficulty multiplier with override
- Soil cache
- Soil map unit overlay toggle
- Configurable rate card in settings

Ship: Soil data auto-populates and adjusts pricing.

Phase 3: Multi-Source Satellite and AI Density (3 to 4 weeks)

- NAIP imagery fetch (natural color + NDVI)
- Sentinel-2 NDVI fetch via Sentinel Hub
- Python AI microservice (FastAPI on Railway)
- NDVI based vegetation classification (rule-based Phase 1)
- Cedar density heatmap generation
- Density score integration into bid engine
- Heatmap overlay toggle on map
- Winter/summer seasonal comparison for cedar detection

Ship: AI density analysis runs on each polygon and feeds into pricing.

Phase 4: PDF Generation (1 to 2 weeks)

- PDF render route with cover, AI analysis page, pasture details, summary
- Puppeteer integration
- Map screenshot capture
- Heatmap and 3D screenshots in PDF
- Branded template with company info
- Version tracking
- Email PDF option

Ship: Professional PDF bids with AI analysis visuals.

Phase 5: 3D Visualization (2 to 3 weeks)

- Mapbox GL JS 3D terrain

- Threebox integration for 3D objects
- Procedural tree models (cedar cone, oak sphere)
- Place trees at AI detected positions
- Camera orbit controls
- Level of detail / instanced rendering for performance
- 3D screenshot capture for PDF
- Toggle between 2D and 3D views

Ship: God's eye 3D view with modeled trees.

Phase 6: Feedback Loop and Self-Improvement (2 to 3 weeks)

- Post job review interface
- Calibration records table
- Prediction engine (kNN then gradient boosting)
- Model retraining pipeline
- Accuracy dashboard
- Before/after photo upload
- Error trend visualization
- Model version tracking

Ship: System learns from every completed job.

Phase 7: Tree Detection ML Model (4 to 8 weeks, ongoing)

- Collect labeled training data from first 10 to 20 jobs
- Set up Label Studio for annotation workflow
- Train U-Net segmentation model for tree detection
- Individual tree counting and size estimation
- Integrate ML predictions into density scoring
- A/B test ML predictions vs rule based
- Continuous retraining as data grows

Ship: Automated tree counting and sizing from satellite imagery.

Phase 8: Polish and Scale (ongoing)

- Client management portal
- QuickBooks/accounting integration
- Equipment tracking
- GPS track recording for auto polygon creation
- Historical change detection (cedar encroachment over time)
- Google Earth Engine integration for deep archive analysis
- Multi company white label option

---

16. Open Questions for Cactus Creek

1. **Rate card:** Exact $/acre by vegetation type and density
2. **Equipment:** What machines do they run? (Brand/model of mulcher, dozer, chainsaw crew size)
3. **Disposal methods:** Primary preference hierarchy
4. **Burn permits:** County process, cost, who handles it
5. **Logo and brand:** SVG or high res PNG, brand colors
6. **Terms and conditions:** Existing T&C language or need new
7. **Insurance info:** What to show on bids
8. **Job size range:** Smallest they'll take, largest they've done
9. **Multi user:** Just owner, or crew leads too?
10. **Existing website/domain:** Subdomain situation
11. **Competitors:** Who bids against them, what do competitor bids look like
12. **Payment terms:** Net 30, deposit %, progress payments
13. **Historical data:** Do they have records from past jobs (acreage, type, time taken, cost)? Even rough notes would help seed the feedback model.
14. **Drone:** Do they have or would they buy a drone? Drone imagery at 1cm resolution would dramatically accelerate the ML model training.
15. **Seasonal variation:** Do rates change seasonally? Winter vs summer clearing preferences?

---

17. Appendices

Appendix A: USDA SDA API Reference

Endpoint	URL
REST Query	`POST https://SDMDataAccess.sc.egov.usda.gov/Tabular/post.rest`
WMS Map Tiles	`https://SDMDataAccess.sc.egov.usda.gov/Spatial/SDM.wms`
WFS Features	`https://SDMDataAccess.sc.egov.usda.gov/Spatial/SDMWGS84Geographic.wfs`
Help / Docs	`https://sdmdataaccess.nrcs.usda.gov/WebServiceHelp.aspx`
Query Examples	`https://sdmdataaccess.nrcs.usda.gov/QueryHelp.aspx`

Appendix B: UC Davis SoilWeb Endpoints

Endpoint	URL
SoilWeb Map App	`https://casoilresource.lawr.ucdavis.edu/gmap/`
Soil Data Explorer	`https://casoilresource.lawr.ucdavis.edu/sde/?series={SERIES_NAME}`
Soil Properties Grid	`https://casoilresource.lawr.ucdavis.edu/soil-properties/`
Series Extent Explorer	`https://casoilresource.lawr.ucdavis.edu/see/`

Appendix C: Satellite Imagery Endpoints

Source	Endpoint	Resolution	Cost	Bands

NAIP	`imagery.nationalmap.gov/arcgis/rest/services/USGSNAIPImagery/ImageServer`	0.6m	Free	4 (RGB+NIR)
NAIP (Texas)	`imagery.geographic.texas.gov/server/rest/services/NAIP/`	0.6m	Free	4
Sentinel-2	`services.sentinel-hub.com/api/v1/process`	10m	Free tier	13
Sentinel-2 AWS	`registry.opendata.aws/sentinel-2`	10m	Free	13
Mapbox Satellite	Built into Mapbox GL JS	Sub-meter	Mapbox plan	RGB
Google Earth Engine	`code.earthengine.google.com`	Various	Free (non-commercial)	Various
Planet Labs	`api.planet.com`	3m daily	$4k to $10k/yr	4+
USGS 3DEP (elevation)	`epqs.nationalmap.gov/v1/json`	1m to 10m	Free	DEM

Appendix D: Kerrville Area Soil Series Reference

Series	Slope	Depth	Rock	Clearing Difficulty
Tarrant	1 to 8%	Very shallow (6 to 20 in)	Very high	Hard. Premium pricing.
Brackett	5 to 40%	Shallow (10 to 20 in)	High	Steep. Erosion risk.
Eckrant	1 to 8%	Very shallow (6 to 14 in)	Very high	Worst case. Limestone surface.
Real	1 to 8%	Shallow (14 to 20 in)	High	Moderate to hard.
Comfort	0 to 5%	Shallow (14 to 20 in)	Moderate	Average Hill Country.
Doss	1 to 5%	Shallow (10 to 20 in)	Moderate	Moderate. Soft limestone.
Krum	0 to 3%	Deep (60+ in)	Low	Easiest. Watch for mud.
Purves	1 to 5%	Very shallow (6 to 18 in)	High	Similar to Tarrant.

Appendix E: NDVI Reference Values for Hill Country

Cover Type	Summer NDVI	Winter NDVI	Seasonal Ratio
Dense cedar	0.55 to 0.75	0.50 to 0.70	0.85 to 0.95
Live oak canopy	0.50 to 0.70	0.25 to 0.45	0.45 to 0.65
Post oak canopy	0.55 to 0.70	0.15 to 0.30	0.25 to 0.45
Native grass (healthy)	0.40 to 0.65	0.10 to 0.25	0.20 to 0.35
Bare ground / caliche	0.05 to 0.15	0.05 to 0.15	~1.0 (no change)
Rock outcrop	-0.05 to 0.10	-0.05 to 0.10	~1.0

These values should be calibrated with ground truth from the first 5 to 10 completed jobs.

End of planning document v2. The satellite AI pipeline and feedback loop are the differentiators that make this more than a bid calculator. Start with Phase 1 (basic map + bid), add AI in Phase 3, and let the feedback loop compound accuracy over time.